IPA ASSIGNMENT

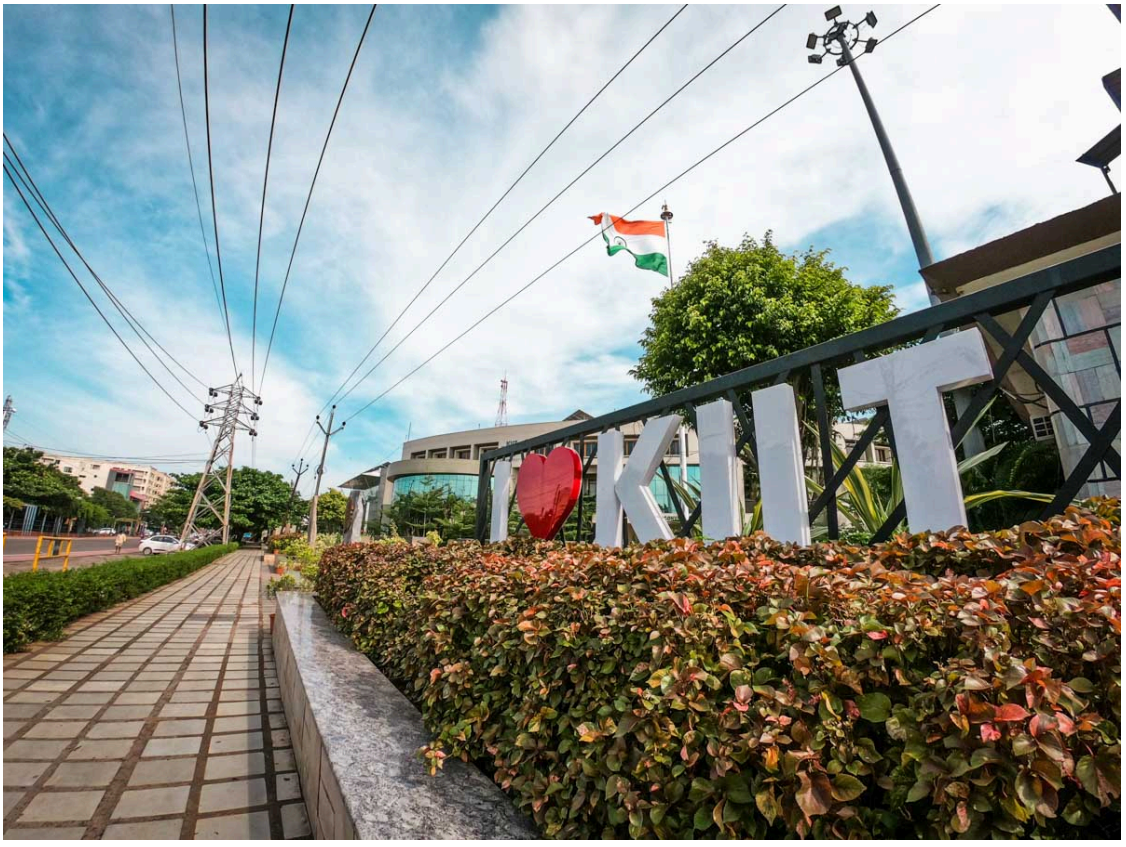
Name- Sanvi Shan
Roll Number-24155797
IPA-12

Software: Python 3.14.3,OpenCV (cv2) 5.0.0, NumPy 2.5.4, Matplotlib 3.11.2

Note: All processing was done in Python using OpenCV for image reading, grayscale conversion, histogram equalization, Gaussian filtering and Sobel operators; NumPy for array maths (negative, gamma, contrast stretching); and Matplotlib for displaying images and histograms. The same grayscale image is the input to Tasks 3 to 6, and each technique is applied to it separately. All grayscale images are displayed with the same intensity scale (0 to 255) and the same size.

TASK 1 : ORIGINAL PHOTOGRAPH

Original Photograph (RGB)



Question) Display your photograph and use your code to find its width, height, number of color channels, and file size in KB or MB. Write the dimensions as width × height.
Solution) **CODE-**

```
import os

import cv2

import numpy as np

import matplotlib.pyplot as plt



PATH = "photo.png"



# %% Task 1: original photograph

bgr = cv2.imread(PATH)                          # OpenCV loads as BGR

rgb = cv2.cvtColor(bgr, cv2.COLOR_BGR2RGB)    # convert to RGB for display

h, w, c = rgb.shape

size_kb = os.path.getsize(PATH) / 1024



print("Image shape (H, W, C):", rgb.shape)

print(f"Dimensions (width x height): {w} x {h}")

print("Number of colour channels:", c)

print(f"File size: {size_kb:.1f} KB ({size_kb/1024:.2f} MB)")



plt.figure(figsize=(7, 5.25))

plt.imshow(rgb); plt.title("Original photograph (RGB)"); plt.axis("off")

plt.show()
```

OUTPUT-

```
✓ bgr = cv2.imread(PATH) # OpenCV loads as BGR …

Image shape (H, W, C): (900, 1200, 3)
Dimensions (width x height): 1200 x 900
Number of colour channels: 3
File size: 1641.4 KB (1.60 MB)
```



Observation: The image is 1200 pixels wide and 900 pixels high with 3 colour channels (shape 900, 1200, 3), and the file size is about 1641 KB (1.60 MB).

TASK 2 :CONVERT TO GRAYSCALE

Question) Convert your photograph to grayscale. Show the color and grayscale images side by side. Briefly explain what changes and what information is still visible.

Solution) **Technique**: Grayscale conversion replaces the three colour channels with one intensity (luminance) channel, using a weighted sum Y = 0.299 R + 0.587 G + 0.114 B. Green is weighted most because the eye is most sensitive to it.

CODE-

```
gray = cv2.cvtColor(bgr, cv2.COLOR_BGR2GRAY)

print("Gray shape:", gray.shape, "| mean: %.1f" % gray.mean())



fig, ax = plt.subplots(1, 2, figsize=(11, 4.2))

ax[0].imshow(rgb);                                ax[0].set_title("Colour (RGB)")

ax[1].imshow(gray, cmap="gray", vmin=0, vmax=255); ax[1].set_title("Grayscale")

for a in ax: a.axis("off")

plt.show()
```

OUTPUT-



Observation. The data drops from 3 channels to 1, so all colour (blue sky, red heart, green and orange flag, reddish leaves) is gone and objects that differ mainly in colour, such as the red heart against the green leaves, become harder to tell apart. The brightness structure is still fully visible: the clouds, the white letters, the pavement tiles, the hedge texture and the thin wires are all still clear, and the mean intensity is about 143.

TASK 3 : NEGATIVE AND GAMMA TRANSFORMATIONS

Question) Create a negative of the grayscale image. Then apply gamma correction with gamma values of 0.5, 1.0, and 2.0. For gamma correction, first scale the pixel values to the range [0, 1] and use s = r^gamma. Show the results and explain how each gamma value affects brightness.

Solution) **Technique**: The negative is s = 255 − r, which flips dark and bright. Gamma correction first scales the pixels to [0, 1] and then applies $s = r^{\gamma}$. Gamma below 1 brightens, gamma above 1 darkens, and gamma = 1.0 leaves the image unchanged. Settings: gamma = 0.5, 1.0 and 2.0.

CODE-

```
                                negative = 255 - gray



                            def gamma_correct(img, gamma):

                    r = img.astype(np.float64) / 255.0      # scale to [0, 1]

                     s = np.power(r, gamma)                  # s = r ^ gamma

                           return np.uint8(np.clip(s * 255.0, 0, 255))



                            g05 = gamma_correct(gray, 0.5)

                            g10 = gamma_correct(gray, 1.0)

                            g20 = gamma_correct(gray, 2.0)



                    for name, im in [("gray", gray), ("negative", negative),

                          ("gamma 0.5", g05), ("gamma 1.0", g10), ("gamma 2.0", g20)]:

                      print(f"{name:10s} mean intensity = {im.mean():6.1f}")



                            imgs   = [gray, negative, g05, g10, g20]

titles = ["Grayscale (input)", "Negative", "Gamma = 0.5", "Gamma = 1.0", "Gamma =
                                        2.0"]
```

```
fig, ax = plt.subplots(2, 3, figsize=(12, 6.6))

for a, im, t in zip(ax.ravel(), imgs, titles):

a.imshow(im, cmap="gray", vmin=0, vmax=255); a.set_title(t); a.axis("off")

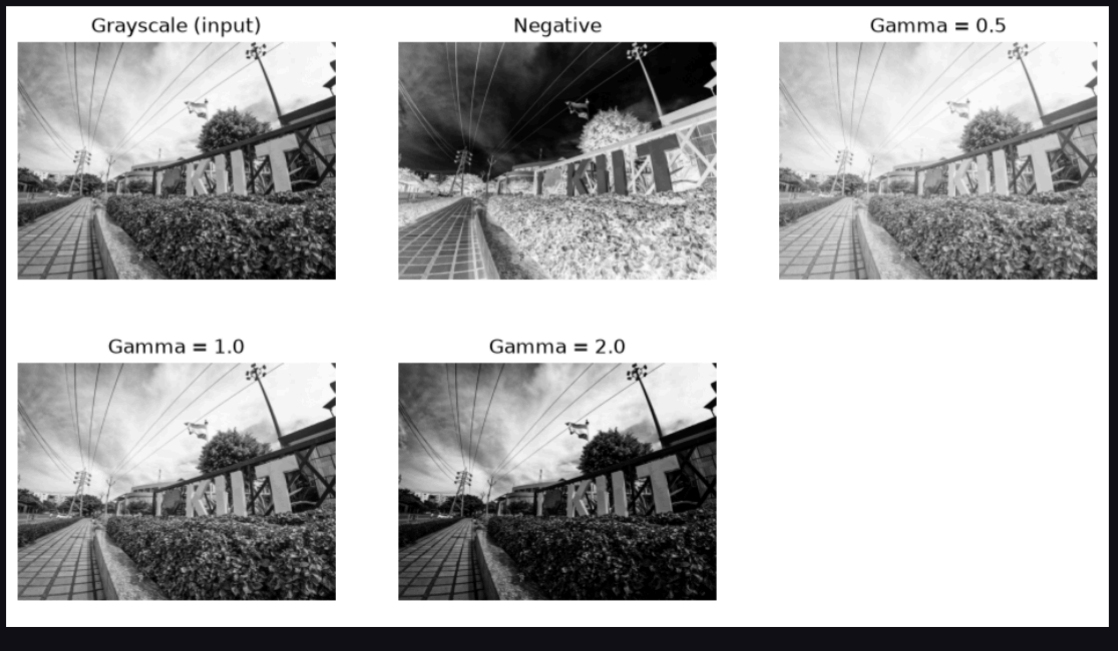
ax[1, 2].axis("off")

plt.show()
```

OUTPUT-

```
✓ negative = 255 - gray ···

gray       mean intensity =  143.0
negative   mean intensity =  112.0
gamma 0.5  mean intensity =  180.9
gamma 1.0  mean intensity =  143.0
gamma 2.0  mean intensity =  103.2
```



Observation: The negative turns the bright sky dark and the dark hedge light, like a photographic negative, so details can be seen in a different way. Gamma 0.5 raises the mean intensity from 143.0 to 180.9: dark regions such as the shaded hedge and fence are lifted and easier to read, but the bright sky is pushed towards white and the cloud contrast is reduced. Gamma 1.0 is identical to the input (mean 143.0). Gamma 2.0 lowers the mean to 103.2: the clouds look richer and the sky less washed out, but the dark hedge and shadows are crushed and lose detail.

TASK 4 : IMPROVE CONTRAST

Question) Apply linear contrast stretching and global histogram equalization separately to the grayscale image. Show each result beside the input. Plot the histogram of the input and of each result using the same intensity range.Are the darker and brighter details easier to see? Explain your observations.

Solution) **Technique**: Linear contrast stretching maps a chosen input range [low, high] linearly to the full range 0 to 255. Global histogram equalization remaps intensities so that the histogram becomes as flat as possible, using the cumulative distribution of the whole image. Settings: stretch limits are the 1st and 99th percentiles (7 and 252); equalization uses `cv2.equalizeHist`; the histograms use 64 bins over 0 to 255 with the same y-axis range.

CODE-

```
# %% Task 4: contrast stretching and histogram equalization

low, high = np.percentile(gray, (1, 99))

stretch = np.clip((gray.astype(np.float64) - low) / (high - low), 0, 1)

stretch = np.uint8(stretch * 255)

print(f"Stretch limits: low = {low:.0f}, high = {high:.0f}")



equal = cv2.equalizeHist(gray)



imgs   = [gray, stretch, equal]

titles = ["Input (gray)", "Contrast stretching", "Histogram equalization"]

fig, ax = plt.subplots(2, 3, figsize=(13, 7), gridspec_kw={"height_ratios": [3,
2]})

hist_max = 0

for i, (im, t) in enumerate(zip(imgs, titles)):

   ax[0, i].imshow(im, cmap="gray", vmin=0, vmax=255)

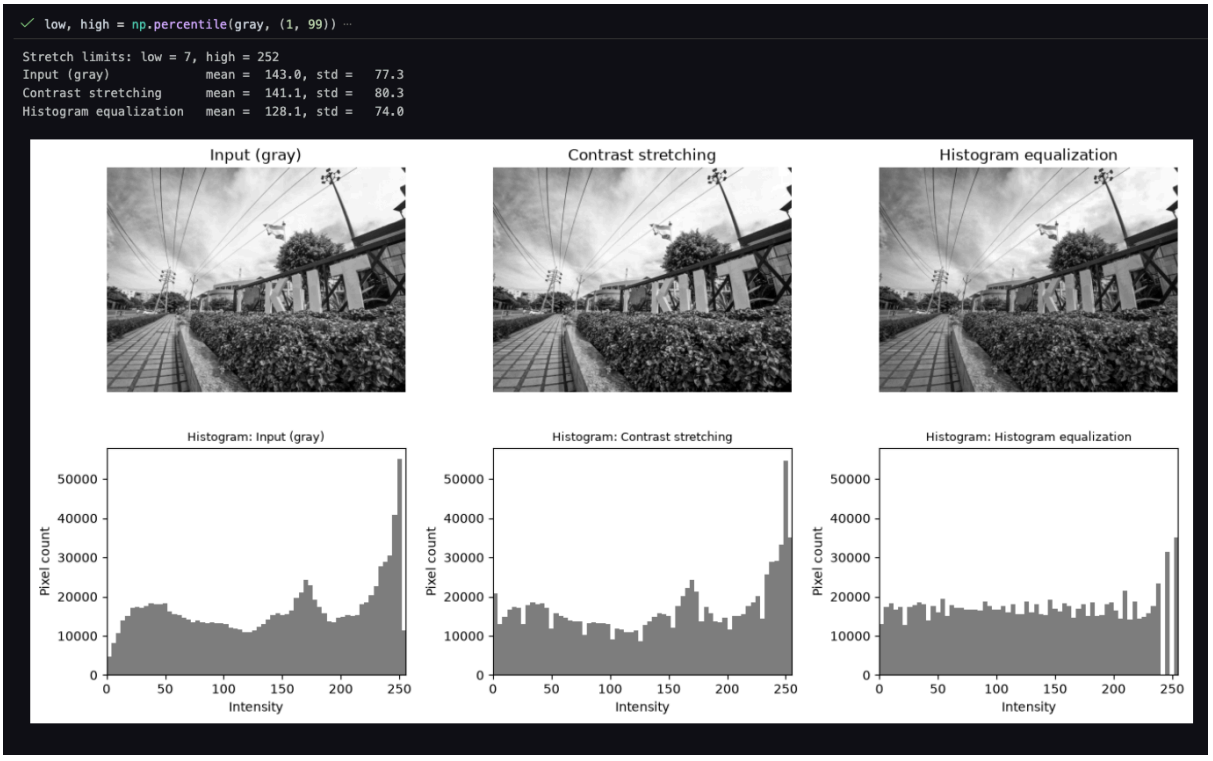
   ax[0, i].set_title(t); ax[0, i].axis("off")

   cnt, _, _ = ax[1, i].hist(im.ravel(), bins=64, range=(0, 256), color="gray")

   hist_max = max(hist_max, cnt.max())
```

```
    ax[1, i].set_xlim(0, 255)
    ax[1, i].set_xlabel("Intensity"); ax[1, i].set_ylabel("Pixel count")
    ax[1, i].set_title("Histogram: " + t, fontsize=9)
    print(f"{t:24s} mean = {im.mean():6.1f}, std = {im.std():6.1f}")
for i in range(3): ax[1, i].set_ylim(0, hist_max * 1.05)    # same y-scale
plt.tight_layout(); plt.show()
```

OUTPUT-



Observation. This photograph already uses almost the whole range (minimum 0, maximum 255), so contrast stretching changes it very little: the limits of 7 and 252 only move the ends slightly, and the histogram keeps the same shape while the standard deviation rises a little (77.3 to 80.3). Histogram equalization changes the shape much more: the histogram becomes far flatter across the range, the large block of pixels near the bright end is spread out, and the mean drops from 143.0 to 128.1. Details in the darker and mid-tone parts are generally easier to see after equalization, while stretching gives a result that looks natural but almost the same as the input.

TASK 5 : APPLY GAUSSIAN SMOOTHING

Question) Smooth the grayscale image using Gaussian filters with 3 × 3 and 7 × 7 kernels. Mention the standard deviation used for each filter. Compare the results with the input. Describe how much blur you notice and whether any fine details are lost.

Solution) **Technique:** A Gaussian filter replaces each pixel by a weighted average of its neighbours, with weights that follow a Gaussian curve. It reduces noise and fine detail. Settings: 3 × 3 kernel with standard deviation sigma = 0.8, and 7 × 7 kernel with sigma = 1.4.

CODE-

```
blur3 = cv2.GaussianBlur(gray, (3, 3), sigmaX=0.8)

blur7 = cv2.GaussianBlur(gray, (7, 7), sigmaX=1.4)



imgs   = [gray, blur3, blur7]

titles = ["Input (gray)", "Gaussian 3x3 (sigma=0.8)", "Gaussian 7x7 (sigma=1.4)"]

fig, ax = plt.subplots(1, 3, figsize=(14, 3.9))

for a, im, t in zip(ax, imgs, titles):

a.imshow(im, cmap="gray", vmin=0, vmax=255); a.set_title(t); a.axis("off")

plt.tight_layout(); plt.show()



# zoomed crop to compare fine detail (hedge leaves)

y0, y1, x0, x1 = 560, 760, 560, 860

fig, ax = plt.subplots(1, 3, figsize=(14, 3.7))

for a, im, t in zip(ax, imgs, titles):

a.imshow(im[y0:y1, x0:x1], cmap="gray", vmin=0, vmax=255)

a.set_title("Zoom: " + t, fontsize=9); a.axis("off")

plt.tight_layout(); plt.show()
```

OUTPUT-



Observation. The 3 × 3 filter gives only a slight softening: the overall scene looks the same and most fine detail, including leaf edges and wires, is still visible. The 7 × 7 filter gives a clearly visible blur: in the zoomed crop the leaf outlines and veins merge into soft patches, the pavement tile lines and thin wires are softened, and the letters lose crisp edges. The Laplacian variance (a sharpness measure) drops from 2083 to 307.5 and to 41.8, which supports this: a larger kernel removes more fine detail.

TASK 6 : DETECT EDGES (SOBEL)

Question) Use the Sobel operators to find intensity changes along the horizontal and vertical directions. Show both responses and the combined gradient magnitude. Describe the object boundaries you can see. If you rescale the responses for display, mention how you did it.

Solution) **Technique:** The Sobel operators estimate the intensity gradient. Gx (derivative along x) responds to vertical edges and Gy (derivative along y) responds to horizontal edges. The gradient magnitude is $\sqrt{G_x^2 + G_y^2}$. Settings: 3 × 3 Sobel kernel, computed in 64-bit float so negative responses are kept.

Display scaling: For display, I took the absolute value of Gx and Gy, divided Gx, Gy and the magnitude by the same maximum value (the maximum magnitude, 956), multiplied by 255, and showed them on a 0 to 255 scale. Using one common divisor keeps the three images comparable.

CODE-

```
# %% Task 6: Sobel edge detection

gx = cv2.Sobel(gray, cv2.CV_64F, 1, 0, ksize=3)   # horizontal gradient

gy = cv2.Sobel(gray, cv2.CV_64F, 0, 1, ksize=3)   # vertical gradient

mag = np.sqrt(gx**2 + gy**2)                      # gradient magnitude



m = mag.max()                                     # same divisor for all three

gx_disp  = np.uint8(np.clip(np.abs(gx) / m * 255, 0, 255))

gy_disp  = np.uint8(np.clip(np.abs(gy) / m * 255, 0, 255))

mag_disp = np.uint8(np.clip(mag / m * 255, 0, 255))



imgs   = [gray, gx_disp, gy_disp, mag_disp]

titles = ["Input (gray)", "Sobel Gx", "Sobel Gy", "Gradient magnitude"]

fig, ax = plt.subplots(2, 2, figsize=(11, 8.2))

for a, im, t in zip(ax.ravel(), imgs, titles):

   a.imshow(im, cmap="gray", vmin=0, vmax=255); a.set_title(t); a.axis("off")
```

```
plt.tight_layout(); plt.show()
```

OUTPUT-



Observation. Gx highlights vertical structures such as the sides of the letters K, I and T, the lamp post, the tower and the pavement joints running towards the vanishing point. Gy highlights horizontal structures such as the top and bottom rails of the fence, the building roof lines and the tile rows on the pavement. The combined magnitude shows the full outlines of the letters, the heart, the flag, the power lines, the tower, the building and the tree. Smooth areas such as most of the sky and the plain letter faces stay dark, while the hedge produces many small edge responses because of its leafy texture.

REFLECTION-

1. Which technique was most useful for my photograph, and why?
Histogram equalization was the most useful for improving visibility, because the photograph has a large bright sky region and darker hedge and pavement, and equalization spreads the intensities so mid-tone and dark detail is easier to see. Sobel edge detection was also very informative for showing the structure of the scene (wires, letters, fence, tiles).

2. How did gamma correction differ from contrast stretching and histogram equalization?
Gamma correction uses a fixed power curve chosen by me: gamma below 1 lifts dark regions and gamma above 1 darkens the image, and it does not look at the image content. Contrast stretching is a straight-line mapping that depends only on the minimum and maximum (or percentile) values, so it has little effect when the image already uses the full range, as here. Histogram equalization is automatic and depends on the whole histogram, so it changes the intensity distribution the most.

3. How did the Gaussian kernel size affect fine details and edges?
The 3 × 3 kernel (sigma 0.8) softened the image only slightly. The 7 × 7 kernel (sigma 1.4) blurred much more: leaf texture, tile lines and thin wires lost sharpness and edges became soft. The larger the kernel, the more fine detail is lost, which is also why smoothing is often used before edge detection to reduce noise.

4. Which techniques made details easier to see, and which helped me view the image in a different way?
Gamma 0.5 and histogram equalization made dark details easier to see, and gamma 2.0 brought out the cloud detail in the bright sky. The negative image and the Sobel edge maps did not so much improve visibility as let me view the image differently: the negative inverts brightness, and Sobel shows only boundaries and structure. Grayscale conversion removed colour but kept the brightness structure.